

Green AI – Carbon-Aware Training Scheduler

Design and Prototype Documentation

Asmitha Ramesh

November 2025

Abstract

This document describes the design and prototype implementation of the *Green AI – Carbon-Aware Training Scheduler*, a small-scale system that reduces the carbon footprint of machine learning (ML) training workloads by choosing *when* and *where* to run them.

The scheduler reads job specifications (deadline, duration, preferred regions) from YAML, and regional carbon-intensity data from a CSV file. It searches over candidate regions and time windows before each job’s deadline, selects the option with the lowest average carbon intensity, and compares that choice against a baseline “run now in the default region” policy. The system logs baseline and actual emission scores, along with estimated carbon savings, to a results file and launches a training script to simulate a real workload.

The prototype is implemented in Python and organized so that its components can be extended to cloud schedulers and MATLAB-based workflows.

1 Introduction

Large-scale ML training has a measurable energy and carbon cost. In practice, many jobs are launched “as soon as possible” in a default region, even when delaying or relocating the job could significantly reduce emissions at little or no impact to the user.

The *Green AI – Carbon-Aware Training Scheduler* is a lightweight prototype that explores this idea on a smaller scale. It focuses on a simple question:

Given a training job with a deadline and a set of available regions, can we pick a time and region that reduces the job’s carbon footprint compared to running immediately?

This document explains the motivation, architecture, data model, scheduling algorithm, execution flow, and metrics produced by the system.

2 Objectives and Scope

The primary objectives of the prototype are:

- Model ML training jobs with deadlines and estimated durations.
- Consume carbon-intensity data for multiple regions over time.
- Implement a simple scheduling algorithm that minimizes carbon intensity within deadline constraints.
- Quantify the benefit of the carbon-aware schedule versus a naive baseline policy.

- Document the workflow and metrics clearly enough that another engineer can run, modify, and extend the system.

Out of scope for this prototype are real-time integration with cloud APIs, multi-tenant queueing or resource contention, and production-grade cost and priority models.

3 System Overview

At a high level, the system consists of five main components:

1. **Job configuration (JobConfig):** YAML-based definitions of training jobs.
2. **Carbon data source (CarbonSource):** CSV-based carbon-intensity data for each region and timestamp.
3. **Scheduler:** Brute-force search that selects the lowest-carbon region and start time before the job's deadline.
4. **Runner:** Command-line entry point that orchestrates scheduling, logging, and job execution.
5. **Metrics Logger and Analysis:** A CSV log (`results.csv`) and a Jupyter notebook that visualizes baseline vs. carbon-aware emissions.

The implementation runs locally but is organized so that JobConfig, CarbonSource, and Scheduler can be reused in a cloud-based deployment or a MATLAB-integrated workflow.

4 Project Structure

The repository is organized as follows:

```
greenai_scheduler/
  carbon_data/
    carbon_data.csv      # sample carbon intensity data by region
  jobs/
    example_job.yaml     # example job definition
  notebooks/
    analysis.ipynb       # plots: baseline vs carbon-aware
  src/
    __init__.py
    job.py               # JobConfig dataclass + YAML loader
    carbon_source.py     # CarbonSource: loads & queries carbon data
    scheduler.py         # Scheduler: picks region + start time
    runner.py            # CLI entry point: tie everything together
    metrics.py           # metrics logger      results.csv
    utils.py             # reserved for helpers
  docs/
    DESIGN.md            # design overview
    USER_GUIDE.md       # step-by-step usage
  docs/images/          # exported figures from analysis
    savings_pct.png
    baseline_vs_carbonaware.png
  train_model.py         # dummy training script
```

```
requirements.txt
README.md
```

The following sections focus on the core components in the `src/` directory and the data files that drive them.

5 Data Model

5.1 Job Configuration

Training jobs are defined in YAML. A sample configuration is shown below:

```
job_name: resnet_cifar10
script: train_model.py
deadline: "2025-11-20T18:00:00"
job_class: "deadline-flexible"
preferred_regions:
  - us-east
  - us-west
  - eu-north
max_cost_per_hour: 2.0
est_duration_hours: 2.0
default_region: "us-east"
```

The YAML is parsed into a JobConfig dataclass:

```
from dataclasses import dataclass
from datetime import datetime
from typing import List

@dataclass
class JobConfig:
    job_name: str
    script: str
    deadline: datetime
    job_class: str
    preferred_regions: List[str]
    max_cost_per_hour: float
    est_duration_hours: float
    default_region: str
```

Key fields:

- **job_name** – unique identifier for the job; used in logs and plots.
- **script** – path to the training script to execute.
- **deadline** – latest acceptable completion time (ISO 8601 string).
- **preferred_regions** – regions the scheduler is allowed to consider.
- **est_duration_hours** – estimated training duration used for planning.
- **default_region** – region used for the baseline “run now” policy.

Fields such as `job_class` and `max_cost_per_hour` are placeholders for future extensions (e.g. cost-aware or priority-aware scheduling).

5.2 Carbon-Intensity Data

Carbon-intensity data is stored in a CSV file:

```
timestamp,region,carbon_intensity
2025-11-20T10:00:00,us-east,450
2025-11-20T10:00:00,us-west,300
2025-11-20T10:00:00,eu-north,150
2025-11-20T11:00:00,us-east,430
2025-11-20T11:00:00,us-west,280
2025-11-20T11:00:00,eu-north,160
...
```

Columns:

- **timestamp** – time of the measurement.
- **region** – region identifier (e.g. us-east).
- **carbon_intensity** – relative carbon score at that time and region.

The CarbonSource class reads this CSV using pandas and provides:

- `get_series(region)` – filtered time series for a region.
- `avg_intensity_in_window(region, start, duration_hours)` – average intensity for the interval $[start, start + duration]$.

If no samples exist inside the requested window, the implementation falls back to the nearest available datapoint for that region.

6 Scheduling Algorithm

The core scheduling logic resides in `Scheduler.schedule_job`. Conceptually, the algorithm performs a brute-force search over allowed regions and feasible start times.

6.1 Inputs

- `job:` `JobConfig` – job to schedule.
- `now:` `datetime` – current time at scheduling.
- `step_hours:` `float` – step size when scanning candidate start times (default: 1 hour).
- `CarbonSource` – provider for carbon-intensity data.

6.2 Candidate Windows

The latest feasible start time is defined as:

$$\text{latest_start} = \text{deadline} - \text{est_duration_hours}.$$

If $\text{latest_start} < \text{now}$, the job is considered urgent and must start immediately; in this case, the scheduler clamps $\text{latest_start} = \text{now}$.

The scheduler then enumerates candidate start times:

$$t \in \{\text{now}, \text{now} + \text{step_hours}, \dots, \text{latest_start}\}.$$

6.3 Region and Time Search

For each region r in `job.preferred_regions` and for each candidate start time t , the scheduler computes:

$$\text{avg_intensity}(r, t) = \text{avg_intensity_in_window}(r, t, \text{est_duration_hours}).$$

The algorithm maintains the best (lowest) intensity seen so far and the corresponding (r, t) pair.

6.4 Baseline vs. Actual Emissions

To quantify the benefit of the carbon-aware policy, the scheduler compares two scenarios:

1. **Baseline:** run the job immediately in the default region.
2. **Carbon-aware:** run the job in the chosen region and time window.

Let d be `est_duration_hours`. The baseline emission score is:

$$\text{baseline_intensity} = \text{avg_intensity_in_window}(\text{default_region}, \text{now}, d),$$

$$\text{baseline_carbon} = \text{baseline_intensity} \times d.$$

If the chosen region and start time have average intensity `best_avg_intensity`, then:

$$\text{actual_carbon} = \text{best_avg_intensity} \times d.$$

The savings percentage is:

$$\text{savings_pct} = \frac{\text{baseline_carbon} - \text{actual_carbon}}{\text{baseline_carbon}} \times 100.$$

These values are packaged into a `ScheduleDecision` dataclass, which includes the job name, chosen region, timing information, and all computed metrics.

6.5 Complexity

For a single job:

- Let R be the number of preferred regions.
- Let T be the number of candidate time steps.

The time complexity is $O(R \times T)$. Given that the prototype uses small values of R and T , this brute-force strategy is acceptable and easy to reason about.

7 Execution Flow

The main entry point is `src/runner.py`. A typical command is:

```
python -m src.runner --job-config jobs/example_job.yaml \  
                  --carbon-csv carbon_data/carbon_data.csv
```

The execution flow is:

1. **Load job configuration.** The runner calls `load_job_config` to parse the YAML into a `JobConfig` instance.
2. **Load carbon data.** `CarbonSource` is instantiated with the CSV path and loads carbon-intensity values into a pandas `DataFrame`.
3. **Schedule the job.** `Scheduler.schedule_job(job, now)` finds the optimal region and start time, then computes baseline and actual emission scores and `savings_pct`.
4. **Log decision and metrics.** The resulting `ScheduleDecision` is passed to the metrics logger, which appends a row to `results.csv`. If the file does not exist, it is created with a header row.
5. **Run the training script.** The runner invokes the script specified in `job.script` using `subprocess.run`. In the current prototype, this script is `train_model.py`, which simulates a brief training task.
6. **Analyze results.** The Jupyter notebook `notebooks/analysis.ipynb` can be used to load `results.csv` and plot carbon savings and emission comparisons.

8 Example Scenario and Results

8.1 Scenario

For a concrete example:

- Job: `resnet_cifar10`
- Estimated duration: 2 hours
- Preferred regions: `us-east`, `us-west`, `eu-north`
- Default region (baseline): `us-east`
- Carbon-intensity patterns: `us-east` has relatively high intensity, while `eu-north` exhibits much lower intensity in the same time window.

8.2 Logged Metrics

A sample row from `results.csv`:

```
timestamp_logged,job_name,region,start_time,est_end_time,avg_carbon_intensity,  
  baseline_carbon,actual_carbon,savings_pct  
2025-11-17T22:30:53.635687,resnet_cifar10,eu-north,2025-11-17T22  
  :30:53.391644,2025-11-18T00:30:53.391644,150.00,900.00,300.00,66.67
```

Interpretation:

- If the job had run immediately in `us-east`, the baseline emission score would have been 900 units.
- Under the carbon-aware schedule, the job is planned in `eu-north` with a lower average intensity, leading to a score of 300.
- The estimated reduction is approximately 66.7%.

8.3 Visualizations

To make the scheduler’s behavior easier to interpret, the project includes a Jupyter notebook that generates two key figures from `results.csv`. The exported PNG files are stored under `docs/images/` and referenced here.

8.3.1 Savings percentage per run

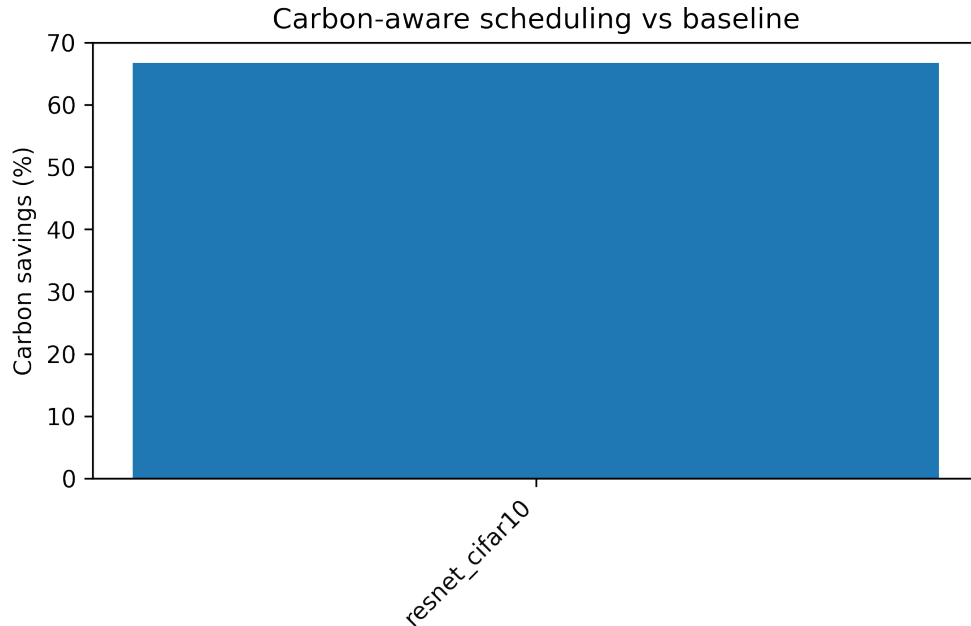


Figure 1: Carbon savings percentage for each scheduled run. Each bar corresponds to one job; the height of the bar represents the relative reduction in the emission score compared to the baseline “run now in the default region” policy. In the example shown, the `resnet_cifar10` job achieves a savings of roughly 66–67%, indicating that the scheduler found a much cleaner region/time window than the naive baseline.

Figure 1 presents a high-level view of how effective the carbon-aware policy is across runs. For each job, the bar shows how much the estimated emission score was reduced by choosing the best available region/time window instead of running immediately in the default region. This makes it very easy to answer questions such as “is the scheduler actually making a difference?” and “which jobs benefit the most from being shifted in time or space?”.

8.3.2 Baseline vs. carbon-aware emission scores

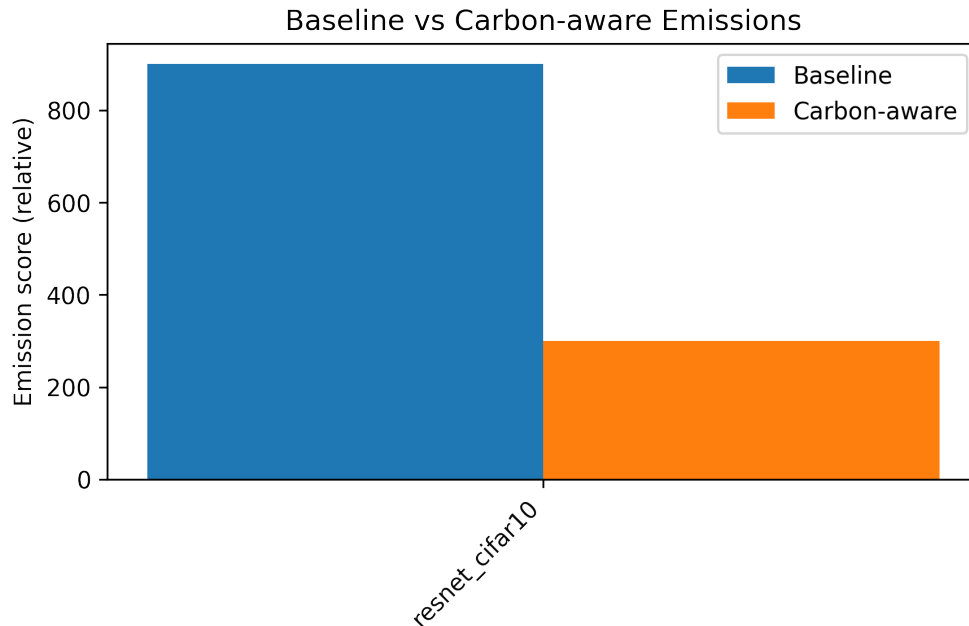


Figure 2: Comparison of baseline and carbon-aware emission scores for each job. The baseline bar shows the estimated emissions if the job is run immediately in the default region; the carbon-aware bar shows the emissions under the chosen schedule. In the illustrated example, the carbon-aware bar for `resnet_cifar10` is approximately one-third of the baseline bar (300 vs. 900), visually reinforcing the 66.7% reduction computed from the metrics.

Figure 2 provides a more detailed view by showing the absolute emission scores for each policy. The left bar for each job corresponds to the baseline “run now” strategy, and the right bar corresponds to the carbon-aware schedule. In the example scenario, the baseline score of 900 units drops to 300 units under the carbon-aware schedule, and the plot makes this difference visually obvious. Together with Figure 1, this helps stakeholders quickly understand not just that the scheduler is finding better options, but also the magnitude of the improvement.

9 Extensibility and Integration Considerations

Although the current implementation uses local files and a dummy Python training script, the architecture is designed with extensibility in mind:

- **Carbon data source.** CarbonSource can be refactored to pull data from external APIs instead of a CSV file. The rest of the system can remain unchanged.
- **Job execution.** The Runner can be adapted to submit jobs to a cloud scheduler or a MATLAB-based pipeline instead of calling a local script. For example, it could generate a MATLAB script that performs training with MATLAB toolboxes and execute it via `matlab -batch`.
- **Multiple jobs and queueing.** The scheduling algorithm can be extended to handle multiple jobs with different priorities, resource requirements, and overlapping deadlines.

- **Cost and policy constraints.** Fields such as `max_cost_per_hour` and `job_class` can be integrated into the decision-making process to balance carbon savings, runtime, and monetary cost.

10 Limitations and Future Work

Current limitations include:

- **Mock carbon data:** the prototype uses a small CSV file with manually defined intensity patterns, rather than live grid data.
- **Single-job scheduling:** jobs are scheduled independently, with no notion of shared resources or contention.
- **Immediate execution:** after scheduling, the job is launched immediately, regardless of the chosen start time. This simplifies the prototype but does not fully model a deferred execution system.
- **Simplified emissions model:** emission scores are proportional to intensity times duration and do not account for hardware differences or power usage.

Future enhancements:

- Integrate with real-time carbon-intensity APIs.
- Implement a job queue and resource model.
- Respect the chosen start time by deferring execution or interacting with an external scheduler.
- Support MATLAB-based training workflows and interactive dashboards to monitor scheduled jobs and savings in real time.

11 Conclusion

The *Green AI – Carbon-Aware Training Scheduler* demonstrates an end-to-end workflow for carbon-aware ML scheduling on a manageable scale:

1. Model training jobs and regional carbon intensity.
2. Search for lower-carbon time windows before a deadline.
3. Compare against a straightforward baseline policy.
4. Log, visualize, and interpret carbon savings.

While the current implementation is intentionally minimal, it provides a clear starting point for experimenting with greener training strategies and can be extended toward real-world deployments that integrate cloud services, live carbon data, and MATLAB-based analytics.